

Atividade Prática EaD: Estruturas de Dados

Busca Sequencial vs. Busca Binária

Prof. Dr. Helberth Borelli

Março de 2026

1 Objetivo

O objetivo desta atividade é realizar uma análise empírica da eficiência de algoritmos de busca em listas lineares sequenciais, comparando o desempenho prático com a teoria de complexidade assintótica $O(n)$ e $O(\log n)$.

2 Análise Empírica vs. Complexidade Teórica

Nesta atividade, utilizaremos a biblioteca `<time.h>` para medir o tempo de execução real (análise empírica). No entanto, é fundamental compreender por que a **Análise de Complexidade** (O) é o padrão ouro na Ciência da Computação:

- **Independência de Hardware:** O tempo de execução em segundos varia se você usa um computador de última geração ou um sistema embarcado (*Edge Computing*). A complexidade $O(n)$ ou $O(\log n)$ nos diz como o algoritmo se comporta independentemente do processador.
- **Previsibilidade de Escala:** A medição de tempo nos mostra o que acontece com 100 mil elementos. A análise de complexidade nos permite prever, com precisão matemática, o que acontecerá quando tivermos bilhões de dados, sem precisar gastar recursos executando o teste.
- **Custo de Crescimento:** Enquanto a busca sequencial cresce linearmente com o tamanho da entrada, a busca binária cresce de forma logarítmica. Medir as comparações ajuda a visualizar que, para o algoritmo, o "esforço" é o que realmente define sua eficiência.

Nota: Ao realizar os testes, observe que para valores pequenos de n , a diferença de tempo pode ser insignificante, mas para valores grandes, a teoria da complexidade se torna a diferença entre um sistema funcional e um sistema travado.

3 Instruções

1. Implemente o código-fonte fornecido na Seção 4.
2. Utilize um compilador C (GCC, Clang ou similar) habilitando as flags de tempo se necessário.
3. Realize os testes para um **alvo inexistente** (ex: -1) para garantir o pior caso.
4. Complete a tabela de resultados e responda às questões discursivas.

4 Código Base

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  #define TAMANHO 100000
6  #define REPETICOES 10000
7
8  int buscaSequencial(int v[], int n, int alvo, long *comp);
9  int buscaBinaria(int v[], int n, int alvo, long *comp);
10
11 int main() {
12     int *vetor = (int *)malloc(TAMANHO * sizeof(int));
13     int alvo = -1;
14     long totalCompSeq = 0, totalCompBin = 0;
15     clock_t inicio, fim;
16
17     for (int i = 0; i < TAMANHO; i++) vetor[i] = i * 2;
18
19     // Teste Sequencial
20     inicio = clock();
21     for(int i = 0; i < REPETICOES; i++) {
22         long c = 0;
23         buscaSequencial(vetor, TAMANHO, alvo, &c);
24         totalCompSeq += c;
25     }
26     fim = clock();
27     printf("Sequencial - Tempo: %.4f s | Media Comp: %ld\n",
28           (double)(fim - inicio) / CLOCKS_PER_SEC, totalCompSeq / REPETICOES);
29
30     // Teste Binário
31     inicio = clock();
32     for(int i = 0; i < REPETICOES; i++) {
33         long c = 0;
34         buscaBinaria(vetor, TAMANHO, alvo, &c);
35         totalCompBin += c;
36     }
37     fim = clock();
38     printf("Binaria - Tempo: %.4f s | Media Comp: %ld\n",
39           (double)(fim - inicio) / CLOCKS_PER_SEC, totalCompBin / REPETICOES);
40
41     free(vetor);
42     return 0;
43 }
```

5 Resultados e Análise

5.1 Tabela de Experimentos

Preencha a tabela abaixo alterando a constante TAMANHO no seu código:

Tamanho (n)	Método	Tempo Total (s)	Média de Comparações
100.000	Sequencial	3.9260 s	100000
100.000	Binária	0.0020 s	16
1.000.000	Sequencial	40.6370 s	141006
1.000.000	Binária	0.0030 s	19

5.2 Questões Discursivas

Responda de forma sucinta e técnica:

1. **Complexidade:** Ao aumentar o vetor em 10 vezes, como se comportou o tempo da Busca Sequencial comparado ao da Busca Binária? Explique com base na teoria $O(n)$ e $O(\log n)$.

A busca sequencial tem a complexidade $O(n)$, e seu tempo de execução cresce linearmente de acordo com o tamanho da entrada. Ao aumentar o valor em 10 vezes (de 100.00 para 1.000.000), o tempo aumentou de 3.9s para 40.6s, confirmando está proporcionalidade. Já a busca binária, com a complexidade $O(\log n)$, cresce de forma logarítmica. As médias de comparações mudou de 16 para 19, tornando essa diferença de tempo de execução quase imperceptível (de 0.002s para 0.003s).

2. **Aplicações:** Em um sistema de *Edge Computing* com restrição de processamento e dados que mudam pouco, qual estratégia de busca você adotaria? Por quê?

Em sistemas de Edge Computing, onde há restrição de processamento, a busca binária é a estratégia ideal se os dados mudam pouco. Pois, o custo de crescimento e as comparações são drasticamente menores. Como os dados mudam pouco, o custo inicial de ordenar o vetor é paga rapidamente pela grande eficiência das buscas subsequentes.

3. **Implementação:** Qual a vantagem de calcular o índice do meio como $\text{esq} + (\text{dir} - \text{esq}) / 2$ em vez do tradicional $(\text{esq} + \text{dir}) / 2$?

A vantagem de usar $\text{esq} + (\text{dir} - \text{esq}) / 2$ em vez de $(\text{esq} + \text{dir}) / 2$ é a prevenção de overflow (estouro de memória). Pois, em linguagens como C, se o valores de esq e dir forem muito grandes, a soma $\text{esq} + \text{dir}$ pode ultrapassar o limite máximo que uma variável inteira pode guardar, gerando um número negativo ou incorreto.

6 Entrega

O relatório deve ser entregue via AVA contendo o arquivo .c e este documento preenchido em PDF até a data estipulada.